

MAST32001 Computational Statistics

Zhi Zhou

2026-10-04

Exercise 1: A Markov chain with circulation

Consider

$$P = \begin{pmatrix} 0.20 & 0.65 & 0.10 & 0.05 \\ 0.05 & 0.20 & 0.65 & 0.10 \\ 0.10 & 0.05 & 0.20 & 0.65 \\ 0.65 & 0.10 & 0.05 & 0.20 \end{pmatrix}$$

```
P_raw <- matrix(  
  c(  
    0.20, 0.65, 0.10, 0.10, 0.05,  
    0.05, 0.20, 0.65, 0.10, 0.10,  
    0.10, 0.05, 0.20, 0.65, 0.10,  
    0.10, 0.10, 0.05, 0.20, 0.65,  
    0.65, 0.10, 0.10, 0.05, 0.20  
  ),  
  nrow = 5,  
  byrow = TRUE  
)  
rowSums(P_raw)
```

```
[1] 1.1 1.1 1.1 1.1 1.1
```

Each row of the printed matrix sums to 1.1, so it is not a valid transition matrix as written. The natural propositional correction is to divide every entry by 1.1. The resulting stochastic matrix is

```
# normalize each row by its row sum
P <- sweep(P_raw, 1, rowSums(P_raw), FUN = "/")
P
```

```
      [,1]      [,2]      [,3]      [,4]      [,5]
[1,] 0.18181818 0.59090909 0.09090909 0.09090909 0.04545455
[2,] 0.04545455 0.18181818 0.59090909 0.09090909 0.09090909
[3,] 0.09090909 0.04545455 0.18181818 0.59090909 0.09090909
[4,] 0.09090909 0.09090909 0.04545455 0.18181818 0.59090909
[5,] 0.59090909 0.09090909 0.09090909 0.04545455 0.18181818
```

Task 1

Find a stationary distribution π , and explain how you obtained it.

Answer:

Let's check whether the resulting matrix is circulant and doubly stochastic. A matrix is circulant if each row is a cyclic right shift of the first row. It is doubly stochastic if every row sum and every column sum equals 1. Based on these definitions, I wrote two helper functions.

```
# Check the circulant structure: each row is a cyclic right shift of the first row.
is_circulant <- function(P, tol = 1e-10) {
  if (!is.matrix(P) || nrow(P) != ncol(P)) {
    return(FALSE)
  }

  n <- nrow(P)

  right_shift <- function(x, k) {
    if (k == 0) {
      x
    } else {
      c(tail(x, k), head(x, -k))
    }
  }

  for (i in seq_len(n)) {
```

```

    expected <- right_shift(P[1, ], i - 1)
    if (!isTRUE(all.equal(P[i, ], expected, tolerance = tol))) {
      return(FALSE)
    }
  }
}

TRUE
}

# Check whether P is doubly stochastic: all rows and all columns sum to 1.
is_doubly_stochastic <- function(P, tol = 1e-12) {
  if (!is.matrix(P) || nrow(P) != ncol(P)) {
    return(FALSE)
  }

  rows_ok <- all(abs(rowSums(P) - 1) < tol)
  columns_ok <- all(abs(colSums(P) - 1) < tol)

  rows_ok && columns_ok
}

```

Now, let's check the resulting matrix.

```

cat("P_raw is circulant:", is_circulant(P_raw), "\n")
cat("P is circulant:", is_circulant(P), "\n")
stopifnot(is_circulant(P))

cat("P_raw is doubly stochastic:",
    is_doubly_stochastic(P_raw),
    "\n")
cat("P is doubly stochastic:", is_doubly_stochastic(P), "\n")
stopifnot(is_doubly_stochastic(P))

```

```

P_raw is circulant: TRUE
P is circulant: TRUE
P_raw is doubly stochastic: FALSE
P is doubly stochastic: TRUE

```

As shown above, the normalized matrix P is circulant and doubly stochastic. Therefore, the uniform distribution is stationary.

```
pi <- rep(1 / ncol(P), ncol(P))
pi
```

```
[1] 0.2 0.2 0.2 0.2 0.2 0.2
```

Let's verify this. To check whether a proposed distribution π is stationary, we need to verify the three defining properties:

1. Every element is nonnegative: $\pi_i \geq 0$.
2. The elements sum to 1: $\sum_i \pi_i = 1$.
3. The stationary equation holds: $\pi P = \pi$

```
is_stationary <- function(pi, P, tol = 1e-12) {
  nonnegative <- all(pi >= 0)
  sums_to_one <- abs(sum(pi) - 1) < tol
  stationary_equation <- all(abs(as.numeric(pi %*% P) - pi) < tol)

  nonnegative && sums_to_one && stationary_equation
}

is_stationary(pi, P)
```

```
[1] TRUE
```

Task 2

Is the chain reversible with respect to π ?

Answer:

Reversibility requires detailed balance:

$$\pi_i P_{ij} = \pi_j P_{ji}.$$

```
# entry (i,j) is pi_i * P_ij
forward_flow <- sweep(P, 1, pi, FUN = "*")

# check whether forward_flow equals its transpose
reversible <- max(abs(forward_flow - t(forward_flow))) < 1e-12

reversible
```

```
[1] FALSE
```

For state 1 and 2:

```
c(pi[1] * P[1, 2], pi[2] * P[2, 1])
```

```
[1] 0.118181818 0.009090909
```

These are unequal, so the chain is not reversible.

Task 3

Is it irreducible and aperiodic? Justify both answers.

Answer:

By definition, a Markov chain is irreducible if every state can be reached from every other state.

```
all(P > 0)
```

```
[1] TRUE
```

In the matrix, rows represent the current state and columns represent the next state. The condition `all(P > 0)` means that every entry P_{ij} is positive. Therefore from any state i , the chain can move directly to any other state j in one step. Since all states can reach one another, the chain is irreducible.

The period of a state is the greatest common divisor of all possible return times to that state, and a state is aperiodic if its period is 1.

```
all(diag(P) > 0)
```

```
[1] TRUE
```

The diagonal entries P_{ii} are the probabilities that the chain stays in the same state for one step. If `all(diag(P) > 0)` is true, then every state has a positive self-loop. Thus, the chain can return the same state in one step. Since one possible return time is 1, the greatest common divisor of all return times is 1, so every state is aperiodic.

Task 4

Starting from state 1, simulate 10,000 steps in R and compare empirical state frequencies with π .

Answer:

For

$$h(x) = I(x \in 1, 2),$$

the running average after m steps is

$$\bar{h}_m = \frac{1}{m} \sum_{t=1}^m h(X_t).$$

Its theoretical limiting is

$$\lim_{m \rightarrow \infty} \bar{h}_m = \mathbb{E}[h(X)] = \sum_{x=1}^5 h(x)\pi_x = \pi_1 + \pi_2.$$

Since the stationary is uniform, so

$$\lim_{m \rightarrow \infty} \bar{h}_m = 0.40$$

```
set.seed(1002)
n_steps <- 10000
current <- 1
x <- integer(n_steps)

for (m in seq_len(n_steps)) {
  current <- sample.int(5, size = 1, prob = P[current, ])
  x[m] <- current
}
```

```
empirical <- prop.table(table(factor(x, levels = 1:5)))
comparison <- rbind(empirical = as.numeric(empirical), stationary = pi)
colnames(comparison) <- paste0("state ", 1:5)
print(round(comparison, 4))
```

	state 1	state 2	state 3	state 4	state 5
empirical	0.1986	0.2016	0.2042	0.1978	0.1978
stationary	0.2000	0.2000	0.2000	0.2000	0.2000

Starting from state 1, I simulated 10,000 transitions using the transition probabilities in P . The empirical frequencies of the five states were all close to 0.2, which agrees with the stationary distribution.

This is expected because the chain is irreducible and aperiodic, so the ergodic theorem guarantees that the empirical state frequencies coverage to the stationary distribution as the number of steps increases.

Task 5

For $h(x) = I(x \in \{1, 2\})$, plot the running average

$$\frac{1}{m} \sum_{t=1}^m h(X_t), \quad m = 1, \dots, 10000,$$

and add its theoretical limiting value. Interpret the plot.

Answer:

```
library(tidyverse)
library(ggpubr)

h <- as.numeric(x %in% c(1, 2))
running_average <- cumsum(h) / seq_len(n_steps)
theoretical_limit <- sum(pi[c(1, 2)])

plot_df <- data.frame(m = seq_len(n_steps), running_average = running_average)

ggplot(plot_df, aes(x = m, y = running_average)) +
```

```

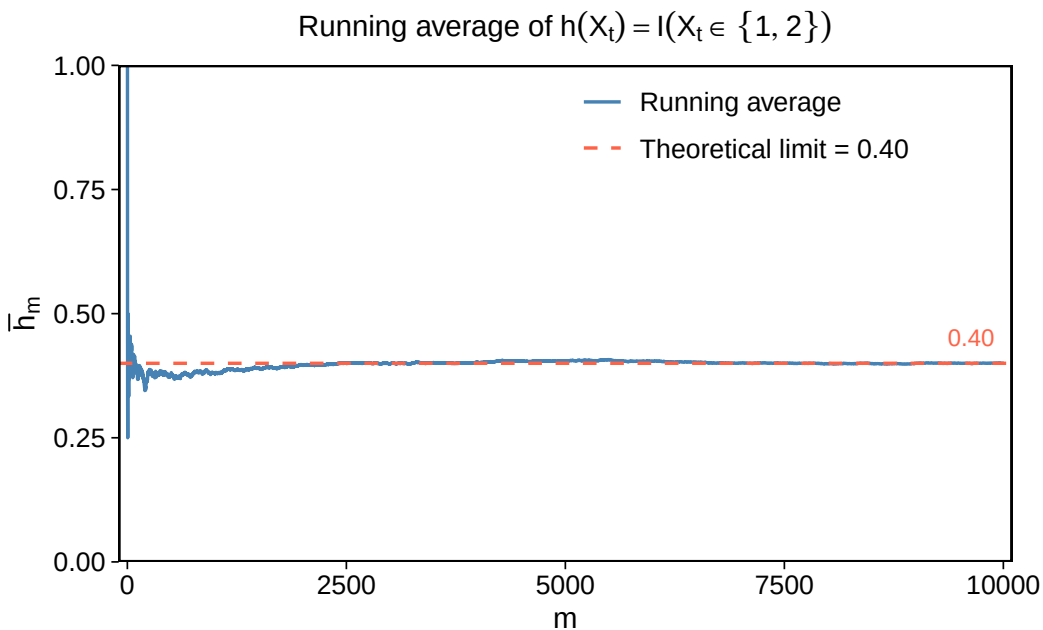
geom_line(aes(colour = "Running average"), linewidth = 0.6) +
geom_hline(
  aes(yintercept = theoretical_limit, colour = "Theoretical limit"),
  linetype = "dashed",
  linewidth = 0.6
) +
annotate(
  "text",
  x = n_steps * 0.99,
  y = theoretical_limit + 0.035,
  label = sprintf("%.2f", theoretical_limit),
  hjust = 1,
  vjust = 0,
  colour = "tomato",
  size = 3.2
) +
scale_colour_manual(
  name = NULL,
  values = c(
    "Running average" = "steelblue",
    "Theoretical limit" = "tomato"
  ),
  labels = c("Running average", "Theoretical limit = 0.40")
) +
scale_x_continuous(expand = expansion(mult = c(0.01, 0.01))) +
scale_y_continuous(limits = c(0, 1), expand = expansion(mult = c(0, 0))) +
labs(
  x = "m",
  y = expression(bar(h)[m]),
  title = expression("Running average of" ~
    h(X[t]) ==
    plain(I)(X[t] %in% group("{", list(
      1, 2
    ), "}"))))
) +
theme_pubr(base_size = 11, border = TRUE) +
theme(
  plot.title = element_text(

```

```

    size = 11,
    face = "plain",
    hjust = 0.5,
    margin = margin(b = 6)
  ),
  axis.text = element_text(colour = "black", size = 10),
  axis.title = element_text(size = 11),
  axis.ticks = element_line(colour = "black", linewidth = 0.3),
  panel.grid = element_blank(),
  legend.position = c(0.70, 0.88),
  legend.background = element_blank(),
  legend.key = element_blank(),
  legend.text = element_text(size = 10),
  plot.margin = margin(10, 12, 8, 8)
)

```



The running average shows random variation during the first few steps, but it gradually stabilizes near the theoretical limiting value 0.40. This follows from the ergodic theorem because the chain is irreducible and aperiodic. As m increases, the effect of the initial state disappears, and the proportion of time spent in states 1 and 2 converges to their total stationary probability.

Exercise 2: Metropolis–Hastings on a transformed scale

Suppose

$$Y|p \sim \text{Binomial}(40, p), \quad Y = 31$$

Define

$$\eta = \log\left(\frac{p}{1-p}\right)$$

and assume directly that

$$\eta \sim N(0, 2^2).$$

Task 1

Derive the log posterior of η , up to an additive constant.

Answer:

The likelihood is

$$Y|p \sim \text{Binomial}(40, p), \quad Y = 31$$

so, up to constants,

$$f(y|p) \propto p^{31}(1-p)^9.$$

With

$$\eta = \log\left(\frac{p}{1-p}\right), \quad p = \frac{e^\eta}{1+e^\eta}, \quad 1-p = \frac{1}{1+e^\eta},$$

the likelihood becomes

$$f(y|p) \propto \left(\frac{e^\eta}{1+e^\eta}\right)^{31} \left(\frac{1}{1+e^\eta}\right)^9 = \frac{e^{31\eta}}{(1+e^\eta)^{40}}.$$

The prior is

$$\eta \sim N(0, 2^2),$$

with log-density

$$\log f(\eta) = -\frac{\eta^2}{8} + C.$$

Because the prior is specified directly on η , no Jacobian is needed. Therefore,

$$f(y|p) \propto \exp(31\eta - 40\log(1+e^\eta) - \frac{\eta^2}{8}),$$

and the log posterior, up to an additive constant, is

$$\log f(y|p) \propto 31\eta - 40\log(1+e^\eta) - \frac{\eta^2}{8} + C.$$

```

# numerically stable computation of log(1 + exp(x))
# direct computation of exp(x) can overflow when x is very large.
log1pexp <- function(x) {
  ifelse(
    x > 0,
    x + log1p(exp(-x)),
    # when x > 0, use log(1+e^x) = x + log(1+e^{-x})
    log1p(exp(x))          # when x <= 0, compute log(1+e^x) directly
  )
}

# unnormalized log posterior of eta
# additive constants do not need to be included because they cancel
# in the Metropolis-Hastings acceptance ratio.
logpost <- function(eta) {
  31 * eta -
  40 * log1pexp(eta) -
  eta^2 / (2 * 2^2)
}

```

Task 2

Implement random-walk Metropolis with

$$\eta^* = \eta^{(t)} + \epsilon, \quad \epsilon \sim N(0, s^2).$$

Answer:

For a random-walk proposal,

$$\eta^* = \eta^{(t)} + \epsilon, \quad \epsilon \sim N(0, s^2).$$

The forward proposal density is

$$q(\eta^* | \eta) = \phi_s(\eta^* - \eta),$$

whereas the density of the reverse move is

$$q(\eta | \eta^*) = \phi_s(\eta - \eta^*).$$

since

$$\eta^* - \eta = \varepsilon, \quad \eta - \eta^* = -\varepsilon,$$

and the zero-centered normal density is symmetric,

$$\phi_s(\varepsilon) = \phi_s(-\varepsilon).$$

Consequently,

$$q(\eta^* | \eta) = q(\eta | \eta^*),$$

so the proposal-density terms cancel in the Metropolis-Hastings ratio. The acceptance probability therefore depends on only the posterior ratio:

$$\alpha(\eta, \eta^*) = \min \left\{ 1, \frac{f(\eta^* | y)}{f(\eta | y)} \right\}.$$

Equivalently, on the log scale,

$$\alpha(\eta, \eta^*) = \min \{1, \exp [\log f(\eta^* | y) - \log f(\eta | y)]\}.$$

The posterior normalizing constant may be omitted because it cancels in this ratio.

```
# random-walk metropolis algorithm
rw_metropolis <- function(n_iter,
                           init = 0,
                           s = 1,
                           burn_in = 0,
                           seed = 1) {
  # Check that the total number of iterations is valid
  if (n_iter < 1) {
    stop("n_iter must be at least 1.")
  }

  # Check that burn-in does not include all iterations
  if (burn_in < 0 || burn_in >= n_iter) {
    stop("burn_in must be between 0 and n_iter - 1.")
  }

  # set the random seed for reproducibility
  set.seed(seed)
```

```

# preallocate a vector to store eta at each iteration
eta_chain <- numeric(n_iter)

# initialize the current parameter value
current <- init

# compute the log posterior at the current value
current_logpost <- logpost(current)

# count how many proposals are accepted
n_accepted <- 0

# start the mcmc iterations
for (i in seq_len(n_iter)) {
  # generate a random-walk proposal:
  # proposal = current + epsilon, where epsilon ~ N(0, s^2)
  proposal <- current + rnorm(1, mean = 0, sd = s)

  # compute the log posterior at the proposed value
  proposal_logpost <- logpost(proposal)

  # computed the log acceptance ratio.
  # the random-walk proposal is symmetric, so no proposal-density ratio is needed.
  log_acceptance_ratio <- proposal_logpost - current_logpost

  # draw a uniform number and make acceptance decision on the log scale.
  if (log(runif(1)) < log_acceptance_ratio) {
    # if accepted, update the current parameter value
    current <- proposal

    # update the log posterior associated with the current value
    current_logpost <- proposal_logpost

    # increase the acceptance counter
    n_accepted <- n_accepted + 1
  }

  # store the current value, whether the proposal was accepted or rejected

```

```

    eta_chain[i] <- current
  }

  # Remove the burn-in iterations and retain the remaining draws
  eta_retained <- eta_chain[(burn_in + 1):n_iter]

  # Return the full chain, retained draws, and sampler information
  list(
    chain = eta_chain,
    retained = eta_retained,
    acceptance_rate = n_accepted / n_iter,
    burn_in = burn_in,
    n_retained = length(eta_retained)
  )
}

```

Task 3

Run 20,000 iterations with $s = 1$, starting at zero, and discard the first 2,000.

Answer:

```

rw_result <- rw_metropolis(
  n_iter = 20000,
  init = 0,
  s = 1,
  burn_in = 2000,
  seed = 10040203
)

rw_result$burn_in

```

```
[1] 2000
```

```
rw_result$n_retained
```

```
[1] 18000
```

```
rw_result$acceptance_rate
```

```
[1] 0.41715
```

```
c(mean_eta = mean(rw_result$retained),  
  sd_eta = sd(rw_result$retained))
```

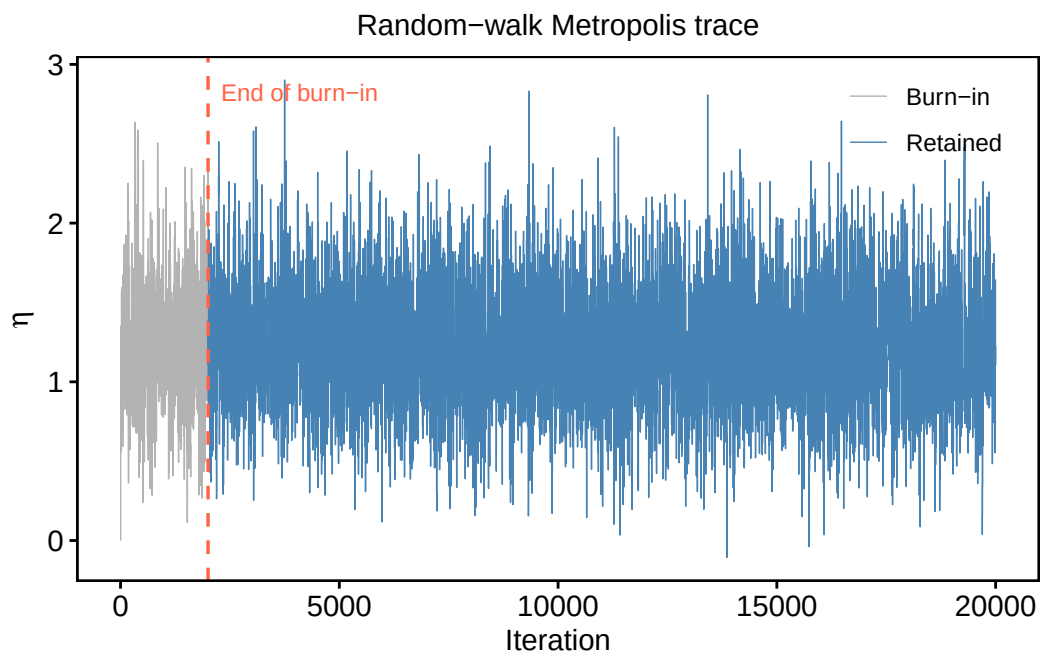
```
mean_eta    sd_eta  
1.2260943 0.3779925
```

```
# trace plot  
trace_df <- data.frame(  
  iteration = seq_len(length(rw_result$chain)),  
  eta = rw_result$chain,  
  phase = factor(rep(  
    c("Burn-in", "Retained"), c(rw_result$burn_in, rw_result$n_retained)  
  ), levels = c("Burn-in", "Retained"))  
)  
  
ggplot(trace_df, aes(x = iteration, y = eta, colour = phase)) +  
  geom_line(linewidth = 0.25) +  
  geom_vline(  
    xintercept = rw_result$burn_in,  
    colour = "tomato",  
    linetype = "dashed",  
    linewidth = 0.6  
  ) +  
  annotate(  
    "text",  
    x = rw_result$burn_in + 300,  
    y = max(trace_df$eta),  
    label = "End of burn-in",  
    hjust = 0,  
    vjust = 1.2,  
    size = 3,  
    colour = "tomato"  
  ) +
```

```

scale_colour_manual(name = NULL,
                    values = c("Burn-in" = "grey70", "Retained" = "steelblue")) +
labs(x = "Iteration", y = expression(eta), title = "Random-walk Metropolis trace") +
theme_pubr(base_size = 11, border = TRUE) +
theme(
  plot.title = element_text(size = 11, hjust = 0.5),
  legend.position = c(0.88, 0.88),
  legend.background = element_blank(),
  legend.key = element_blank()
)

```



Task 4

Transform the retained draws to p and estimate $E(p|y)$ and $P(p > 0.75|y)$.

Answer:

```

# transform
p_post <- plogis(rw_result$retained)
# estimate E(p|y)
mean_p <- mean(p_post)
mean_p

```

```
[1] 0.766543
```

```
# estimate  $P(p > 0.75 | y)$ 
prob_p_gt_0.75 <- mean(p_post > 0.75)
prob_p_gt_0.75
```

```
[1] 0.626
```

```
# plot p_post
p_df <- data.frame(p = p_post)

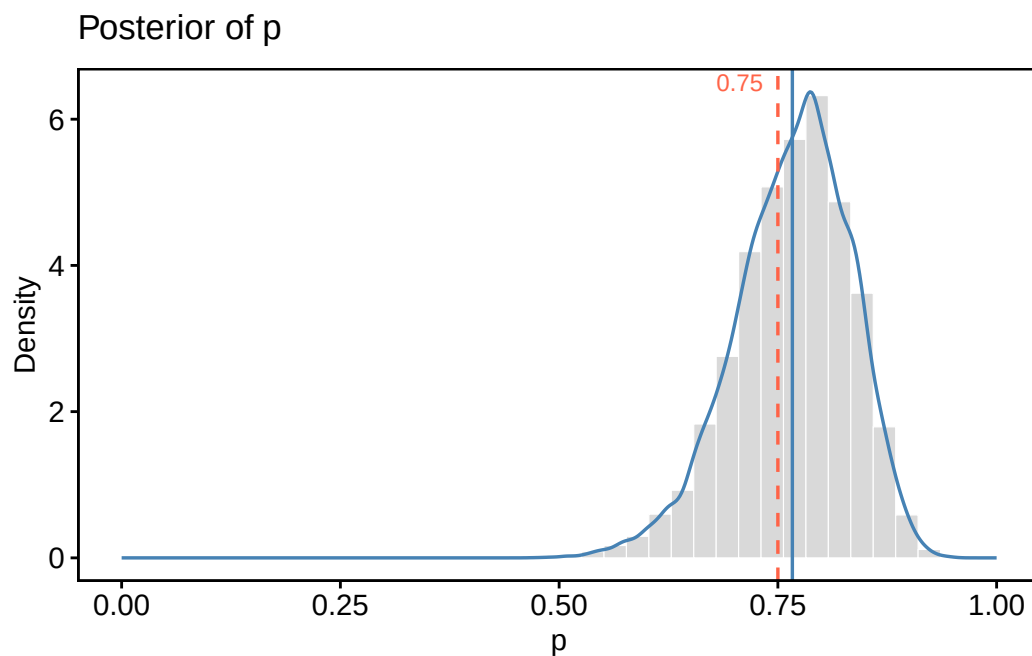
ggplot(p_df, aes(x = p)) +
  geom_histogram(
    aes(y = after_stat(density)),
    bins = 40,
    fill = "grey85",
    colour = "white",
    linewidth = 0.2
  ) +
  geom_density(colour = "steelblue", linewidth = 0.6) +
  geom_vline(
    xintercept = 0.75,
    colour = "tomato",
    linetype = "dashed",
    linewidth = 0.6
  ) +
  annotate(
    "text",
    x = 0.68,
    y = Inf,
    label = "0.75",
    hjust = 0,
    vjust = 1.3,
    size = 3.2,
    colour = "tomato"
  ) +
  geom_vline(xintercept = mean_p,
    colour = "steelblue",
```

```

    linewidth = 0.6) +
scale_x_continuous(limits = c(0, 1)) +
labs(x = "p",
     y = "Density",
     title = expression("Posterior of" ~ p)) +
theme_pubr(base_size = 11, border = TRUE)

```

Warning: Removed 2 rows containing missing values or values outside the scale range (`geom\_bar()`).



Task 5

Explain why the logit scale is convenient compared with a Gaussian random walk directly on p .

Answer:

```

set.seed(10040205)

n_proposals <- 100000
eta_current <- 1.2
p_current <- plogis(eta_current)

```

```

s <- 0.1

# gaussian random walk directly on eta
eta_proposals <- eta_current + rnorm(n_proposals, mean = 0, sd = s)
p_from_logit <- plogis(eta_proposals)

# gaussian random walk directly on p
p_proposals <- p_current + rnorm(n_proposals, mean = 0, sd = s)

# check whether proposed probabilities are outside [0,1]
invalid_p <- p_proposals <= 0 | p_proposals >= 1
invalid_logit <- p_from_logit <= 0 | p_from_logit >= 1

result <- c(
  current_p = p_current,
  invalid_p_scale = mean(invalid_p),
  invalid_logit_scale = mean(invalid_logit)
)

print(round(result, 4))

```

current_p	invalid_p_scale	invalid_logit_scale
0.7685	0.0102	0.0000

This means: on the **p-scale**, even with a relatively small proposal standard deviation $s = 0.1$, about 1% of the proposals are invalid because they fall outside $(0, 1)$; on the **logit scale**, none of the proposals are invalid. The logistic transformation always maps a finite value of η into $(0, 1)$.

The transformation also works for extreme proposals:

```
plogis(c(-100, 100))
```

```
[1] 3.720076e-44 1.000000e+00
```

Approximately 0 and 1, but still strictly within $(0, 1)$.

Exercise 3: Diagnose and repair a sampler

The target is $N(0, 22)$. Use the following sampler.

```
log_target <- function(x)
  dnorm(x, 0, 2, log = TRUE)
rw_normal <- function(initial, n_iter, proposal_sd) {
  x <- numeric(n_iter)
  x[1] <- initial
  acc <- 0
  for (i in 2:n_iter) {
    2
    prop <- rnorm(1, x[i - 1], proposal_sd)
    if (log(runif(1)) <
        log_target(prop) - log_target(x[i - 1])) {
      x[i] <- prop
      acc <- acc + 1
    } else
      x[i] <- x[i - 1]
  }
  list(draws = x, acceptance = acc / (n_iter - 1))
}
```

Task 1

Run four chains of length 2,000 from $-12, -4, 4, 12$ using `proposal_sd = 0.03`.

Answer:

```
set.seed(10040301)

starts <- c(-12, -4, 4, 12)

original_chains <- lapply(starts, function(initial) {
  rw_normal(initial = initial,
            n_iter = 2000,
            proposal_sd = 0.03)
})
```

```
# extract the draws and acceptance rates
original_draws <- do.call(rbind, lapply(original_chains, function(fit)
  fit$draws))

original_acceptance <- sapply(original_chains, function(fit)
  fit$acceptance)

data.frame(
  start = starts,
  acceptance = round(original_acceptance, 3),
  final_value = original_draws[, 2000]
)
```

	start	acceptance	final_value
1	-12	0.968	-10.220755
2	-4	0.989	-3.062730
3	4	0.995	2.545944
4	12	0.967	9.188059

The acceptance rates are very high, but the chains remain close to their initial values, which already suggests poor exploration.

Task 2

Produce suitable diagnostic plots and numerical summaries. Decide yourself what is useful.

Answer:

```
# prepare the trace data
trace_df <- do.call(rbind, lapply(seq_along(starts), function(i) {
  data.frame(
    iteration = 1:2000,
    x = original_chains[[i]]$draws,
    start = factor(starts[i])
  )
}))
```

```

# trace plot
p_trace <- ggplot(trace_df, aes(x = iteration, y = x, colour = start)) +
  annotate(
    "rect",
    xmin = -Inf,
    xmax = Inf,
    ymin = -2,
    ymax = 2,
    fill = "black",
    alpha = 0.06
  ) +
  geom_hline(
    yintercept = 0,
    colour = "black",
    linetype = "dotted",
    linewidth = 0.3
  ) +
  geom_line(linewidth = 0.5) +
  scale_colour_manual(name = "Initial value",
    values = c("springgreen4", "tan2", "steelblue", "tomato")) +
  labs(x = "Iteration", y = "X", title = "Original sampler: proposal SD = 0.03") +
  theme_pubr(base_size = 10, border = TRUE) +
  theme(panel.grid = element_blank())

# density plot
p_density <- ggplot(trace_df, aes(x = x)) +
  geom_histogram(
    aes(
      y = after_stat(density),
      fill = start,
      colour = start
    ),
    bins = 50,
    position = "identity",
    alpha = 0.1,
    linewidth = 0.2
  ) +
  geom_density(aes(colour = start), linewidth = 0.5) +

```

```

stat_function(
  fun = dnorm,
  args = list(mean = 0, sd = 2),
  colour = "black",
  linetype = "dashed",
  linewidth = 0.6,
  inherit.aes = FALSE
) +
scale_fill_manual(name = "Initial value",
                  values = c("springgreen4", "tan2", "steelblue", "tomato")) +
scale_colour_manual(name = "Initial value",
                    values = c("springgreen4", "tan2", "steelblue", "tomato")) +
labs(x = "X", y = "Density") +
theme_pubr(base_size = 10, border = TRUE) +
theme(panel.grid = element_blank())

# numerical summaries
chain_means <- rowMeans(original_draws)
chain_vars <- apply(original_draws, 1, var)
acceptance <- sapply(original_chains, function(fit)
  fit$acceptance)

chain_summary <- data.frame(
  start = starts,
  acceptance = round(acceptance, 3),
  mean = round(chain_means, 3),
  variance = round(chain_vars, 3)
)

chain_summary

```

	start	acceptance	mean	variance
1	-12	0.968	-11.019	0.970
2	-4	0.989	-3.650	0.111
3	4	0.995	2.852	0.272
4	12	0.967	11.176	0.550

```

# Gelman-Rubin R
rhat <- function(chains) {
  chain_means <- rowMeans(chains)
  chain_vars <- apply(chains, 1, var)

  n_iter <- ncol(chains)
  within <- mean(chain_vars)
  between <- n_iter * var(chain_means)

  var_hat <- (1 - 1 / n_iter) * within + between / n_iter
  sqrt(var_hat / within)
}

rhat(original_draws)

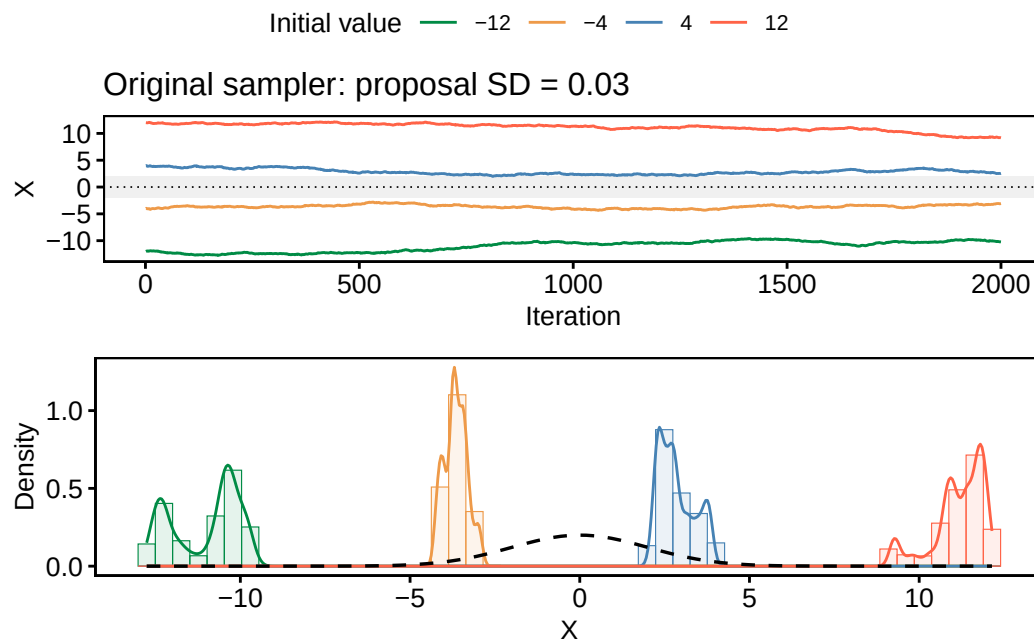
```

```
[1] 13.73345
```

```

# combine the plots
ggarrange(
  p_trace,
  p_density,
  nrow = 2,
  heights = c(1, 1.1),
  common.legend = TRUE,
  legend = "top"
)

```



Task 3

Explain what is wrong and why the acceptance rate alone is misleading.

Answer:

```
# extract draws and acceptance rates
draws <- do.call(
  rbind,
  lapply(original_chains, function(fit) fit$draws)
)

acceptance <- sapply(
  original_chains,
  function(fit) fit$acceptance
)

# compute movement between consecutive iterations
movement <- t(apply(draws, 1, diff))

# lag-1 autocorrelation for each chain
lag1_autocorrelation <- apply(draws, 1, function(x) {
```

```

    cor(x[-length(x)], x[-1])
  })

# standard deviation within each chain
chain_sd <- apply(draws, 1, sd)

# R-hat function
rhat <- function(chains) {
  chain_means <- rowMeans(chains)
  chain_vars <- apply(chains, 1, var)

  n_iter <- ncol(chains)
  within <- mean(chain_vars)
  between <- n_iter * var(chain_means)

  var_hat <- (1 - 1 / n_iter) * within + between / n_iter
  sqrt(var_hat / within)
}

data.frame(
  start = starts,
  acceptance = round(acceptance, 3),
  mean_abs_step = round(mean(abs(movement)), 4),
  lag1 = round(lag1_autocorrelation, 4),
  chain_sd = round(chain_sd, 3)
)

```

	start	acceptance	mean_abs_step	lag1	chain_sd
1	-12	0.968	0.023	0.9996	0.985
2	-4	0.989	0.023	0.9962	0.333
3	4	0.995	0.023	0.9984	0.522
4	12	0.967	0.023	0.9993	0.742

```
rhat(draws)
```

```
[1] 13.73345
```

```
apply(draws, 1, function(x) coda::effectiveSize(x))
```

```
[1] 0.850255 4.853984 2.916416 2.842849
```

The proposal SD is far too small. `acceptance` is close to 1, so almost every proposal is accepted. `mean_abs_step` is only about 0.023, showing that the chain barely moves. `lag1_autocorrelation` is almost 1, meaning consecutive draws contain almost no new information. Therefore the high acceptance rate indicates conservative tiny proposals rather than good exploration. `chain_sd` is far below the target standard deviation 2. the low effective sample size and large $\hat{R}$ demonstrate poor mixing.

Task 4

Modify the simulation to obtain substantially better output. Justify the change and demonstrate the improvement.

Answer:

```
repaired_chains <- lapply(
  starts,
  function(initial) {
    rw_normal(
      initial = initial,
      n_iter = 2000,
      proposal_sd = 4
    )
  }
)

# extract the repaired draws and summaries
repaired_draws <- do.call(
  rbind,
  lapply(repaired_chains, function(fit) fit$draws)
)

repaired_acceptance <- sapply(
  repaired_chains,
```

```

function(fit) fit$acceptance
)

repaired_summary <- data.frame(
  start = starts,
  acceptance = round(repaired_acceptance, 3),
  chain_mean = round(rowMeans(repaired_draws), 3),
  chain_variance = round(apply(repaired_draws, 1, var), 3)
)

repaired_summary

```

	start	acceptance	chain_mean	chain_variance
1	-12	0.510	0.052	4.224
2	-4	0.494	-0.129	4.347
3	4	0.504	-0.108	3.730
4	12	0.504	0.016	4.095

```

# the pooled summaries
c(
  acceptance = mean(repaired_acceptance),
  mean = mean(repaired_draws),
  variance = var(as.vector(repaired_draws)),
  Rhat = rhat(repaired_draws)
)

```

acceptance	mean	variance	Rhat
0.50300150	-0.04223172	4.10342420	1.00073509

```

# prepare repaired-chain data
repaired_df <- do.call(rbind, lapply(seq_along(starts), function(i) {
  data.frame(
    iteration = seq_along(repaired_chains[[i]]$draws),
    x = repaired_chains[[i]]$draws,
    start = factor(starts[i])
  )
}))

```

```

palette <- c("springgreen4", "tan2", "steelblue", "tomato")

# Repaired trace plot
p_repaired_trace <- ggplot(repaired_df,
                           aes(x = iteration, y = x, colour = start)) +
  annotate(
    "rect",
    xmin = -Inf, xmax = Inf,
    ymin = -2, ymax = 2,
    fill = "black", alpha = 0.06
  ) +
  geom_hline(yintercept = 0, colour = "black",
             linetype = "dotted", linewidth = 0.3) +
  geom_line(linewidth = 0.2) +
  scale_colour_manual(name = "Initial value", values = palette) +
  labs(
    x = "Iteration",
    y = "X",
    title = "Repaired sampler: proposal SD = 4"
  ) +
  theme_pubr(base_size = 10, border = TRUE) +
  theme(
    panel.grid = element_blank(),
    plot.title = element_text(size = 11, hjust = 0.5),
    legend.position = "top"
  )

# repaired density plot
p_repaired_density <- ggplot(repaired_df, aes(x = x)) +
  geom_histogram(
    aes(y = after_stat(density),
        fill = start,
        colour = start),
    bins = 50,
    position = "identity",
    alpha = 0.08,
    linewidth = 0.15
  )

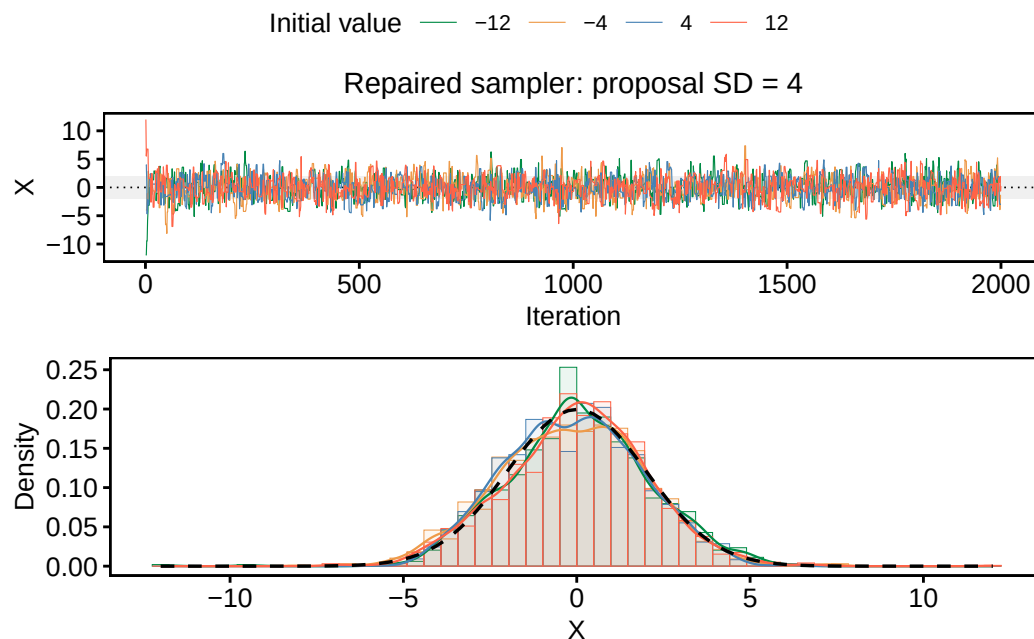
```

```

) +
geom_density(
  aes(colour = start),
  linewidth = 0.4
) +
stat_function(
  fun = dnorm,
  args = list(mean = 0, sd = 2),
  colour = "black",
  linetype = "dashed",
  linewidth = 0.6,
  inherit.aes = FALSE
) +
scale_fill_manual(name = "Initial value", values = palette) +
scale_colour_manual(name = "Initial value", values = palette) +
labs(x = "X", y = "Density") +
theme_pubr(base_size = 10, border = TRUE) +
theme(
  panel.grid = element_blank(),
  legend.position = "top"
)

# combine trace and density into one repaired figure
ggarrange(
  p_repaired_trace,
  p_repaired_density,
  nrow = 2,
  heights = c(1, 1.1),
  common.legend = TRUE,
  legend = "top"
)

```



I repaired the sampler by increasing the proposal standard deviation from 0.03 to 4. The original proposal scale was too small, producing a misleadingly high acceptance rate and very slow movement. The repaired sampler has an acceptance rate around 0.50, overlapping chains, chain variances close to 4, and $\hat{R}$ close to 1. The trace and density plots therefore show substantially better exploration and convergence.

Task 5

Compare estimates of $E(X)$ and $Var(X)$ with their known values.

Answer:

```
burn_in <- 500

repaired_draws <- do.call(
  rbind,
  lapply(repaired_chains, function(fit) fit$draws)
)

retained_draws <- repaired_draws[, (burn_in + 1):2000]
x <- as.vector(retained_draws)

estimated_mean <- mean(x)
```

```

estimated_variance <- var(x)

c(
  estimated_mean = estimated_mean,
  estimated_variance = estimated_variance
)

```

```

estimated_mean estimated_variance
-0.04968363      3.92229096

```

After combining the repaired chains and removing the first 500 iterations, the estimated mean was approximately -0.05 and the estimated variance was approximately 3.92. These are close to the known target values $E(X) = 0$ and $\text{Var}(X) = 4$, confirming that the repaired sampler recovers the target distribution accurately.

Exercise 4: Multimodality

Consider

$$\pi(x) = \frac{1}{2}\phi(x; -5, 1) + \frac{1}{2}\phi(x; 5, 1)$$

Task 1

Implement the log target. Avoid unnecessary numerical instability when evaluating the mixture.

Answer:

Define the two Gaussian log-density components, omitting the common and normal constant:

$$a = -\frac{1}{2}(x + 5)^2, \quad b = -\frac{1}{2}(x - 5)^2$$

To avoid numerical underflow, use the log-sum-exp trick. Let

$$m = \max(a, b)$$

Then

$$\log \pi(x) = m + \log\left[\frac{1}{2}e^{a-m} + \frac{1}{2}e^{b-m}\right] + C$$

where $C = -\frac{1}{2} \log(2\pi)$. The constant can be omitted in Metropolis-Hastings because it cancels.

```

log_target <- function(x) {
  a <- -0.5 * (x + 5)^2
  b <- -0.5 * (x - 5)^2
  m <- max(a, b)

  m + log(0.5 * exp(a - m) +
          0.5 * exp(b - m))
}

# sampler with either a local random walk or an optional mode-jumping proposal
mixture_sampler <- function(initial,
                             n_iter = 15000,
                             proposal_sd = 0.8,
                             jump_prob = 0,
                             jump_sd = 1,
                             seed = 1) {
  set.seed(seed)

  draws <- numeric(n_iter)
  draws[1] <- initial
  current <- initial
  n_accepted <- 0
  n_mode_switches <- 0

  for (i in 2:n_iter) {
    if (jump_prob > 0 && runif(1) < jump_prob) {
      # propose near the opposite mode
      proposal <- -current + rnorm(1, mean = 0, sd = jump_sd)
    } else {
      # ordinary local Gaussian random walk
      proposal <- current + rnorm(1, mean = 0, sd = proposal_sd)
    }

    # both kernels are symmetric, so the acceptance ratio is just the target ratio.
    log_acceptance_ratio <- log_target(proposal) - log_target(current)

    if (log(runif(1)) < log_acceptance_ratio) {
      if (sign(proposal) != sign(current)) {

```

```

    n_mode_switches <- n_mode_switches + 1
  }

  current <- proposal
  n_accepted <- n_accepted + 1
}

draws[i] <- current
}

list(
  draws = draws,
  acceptance = n_accepted / (n_iter - 1),
  mode_switches = n_mode_switches
)
}

```

Task 2

Run four random-walk Metropolis chains of length 15,000 with proposal SD 0.8, starting near both modes.

Answer:

Use two starts near a and two near b :

```

starts <- c(-6, -4, 4, 6)
n_iter <- 15000
burn_in <- 1000

# original local random-walk chains
local_fit <- lapply(seq_along(starts), function(i) {
  mixture_sampler(
    initial = starts[i],
    n_iter = n_iter,
    proposal_sd = 0.8,
    jump_prob = 0,
    seed = 100 + i
  )
})

```

```

    )
  })

data.frame(start = starts, acceptance = round(sapply(local_fit, function(fit)
  fit$acceptance), 3))

```

	start	acceptance
1	-6	0.762
2	-4	0.757
3	4	0.757
4	6	0.761

The acceptance rates look reasonable.

Task 3

Plot the chains and calculate the fraction of retained draws greater than zero for each chain.

Answer:

Discard the first 1,000 iterations as burn-in, then calculate

$$\hat{P}(X > 0) = \frac{1}{N} \sum_t I(X_t > 0)$$

for each chain.

```

burn_in <- 1000
n_iter <- 15000

fraction_positive <- sapply(local_fit, function(fit) {
  mean(fit$draws[(burn_in + 1):n_iter] > 0)
})

fraction_table <- data.frame(start = starts,
                             fraction_positive = round(fraction_positive, 3))

fraction_table

```

```

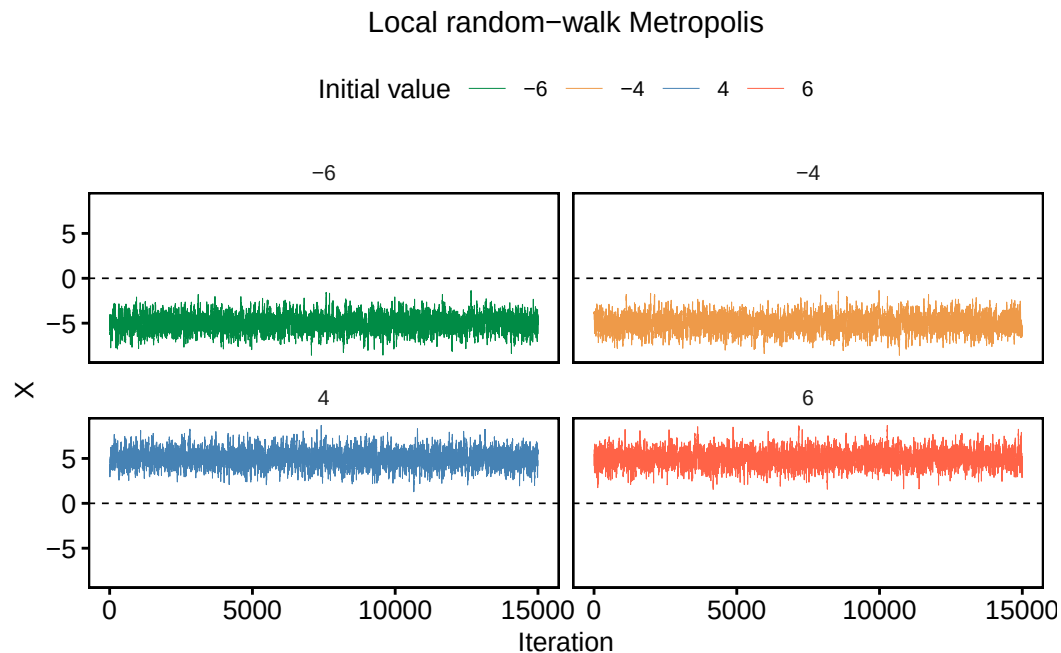
      start fraction_positive
1      -6                  0
2      -4                  0
3       4                  1
4       6                  1

# plot the four chains
trace_df <- do.call(rbind, lapply(seq_along(starts), function(i) {
  data.frame(
    iteration = 1:n_iter,
    x = local_fit[[i]]$draws,
    start = factor(starts[i])
  )
}))

palette <- c("springgreen4", "tan2", "steelblue", "tomato")

p_trace <- ggplot(trace_df, aes(x = iteration, y = x, colour = start)) +
  geom_line(linewidth = 0.15) +
  geom_hline(
    yintercept = 0,
    colour = "black",
    linetype = "dashed",
    linewidth = 0.3
  ) +
  facet_wrap( ~ start, ncol = 2) +
  scale_colour_manual(name = "Initial value", values = palette) +
  labs(x = "Iteration", y = "X", title = "Local random-walk Metropolis") +
  theme_pubr(base_size = 10, border = TRUE) +
  theme(
    panel.grid = element_blank(),
    strip.background = element_blank(),
    plot.title = element_text(size = 11, hjust = 0.5),
    legend.position = "top"
  )
p_trace

```



Because two equally weighted normal components are symmetric around 0, the true target probability is $P(X > 0) = 0.5$. These results show that the local random-walk chains are stable within one mode but do not explore the full bi-modal target. They are trapped near their starting modes.

Task 4

Explain why a single apparently stable trace plot can be seriously misleading here.

Answer:

```
# take one chain started near the left mode
single_chain <- local_fit[[1]]$draws
retained <- single_chain[(burn_in + 1):n_iter]

# summaries of the single chain
single_chain_summary <- data.frame(
  mean = mean(retained),
  variance = var(retained),
  fraction_positive = mean(retained > 0),
  acceptance = local_fit[[1]]$acceptance
)

round(single_chain_summary, 3)
```

	mean	variance	fraction_positive	acceptance
1	-5.013	0.995	0	0.762

This chain looks stable because it fluctuates around -5, has variance close to 1, shows no obvious drift, and has a reasonable acceptance rate. However, it is only sampling a .

```
target_summary <- data.frame(mean = 0,
                              variance = 26,
                              fraction_positive = 0.5)

target_summary
```

	mean	variance	fraction_positive
1	0	26	0.5

The full target has different summaries.

Task 5

Modify the proposal mechanism to improve movement between modes. Demonstrate that your modification helps. More than one solution is possible.

Answer:

One effective modification is to add an explicit **mode-jumping proposal** to the original local random walk.

With probability 0.15, propose near the opposite mode:

$$x^* = -x + \varepsilon_J, \quad \varepsilon_J \sim N(0, 1).$$

Otherwise, use the original local proposal:

$$x^* = x + \varepsilon_L, \quad \varepsilon_L \sim N(0, 0.8^2).$$

The mode-jump kernel is symmetric. If

$$x^* = -x + \varepsilon_J,$$

then the reverse move is

$$x = -x^* + \varepsilon_J,$$

which use the same normal increment. Hence,

$$q(x^* | x) = q(x | x^*),$$

and the acceptance probability remains

$$\alpha(x, x^*) = \min \{1, \exp[\log \pi(x^*) - \log \pi(x)]\}.$$

```
jump_chains <- lapply(seq_along(starts), function(i) {
  mixture_sampler(
    initial = starts[i],
    n_iter = n_iter,
    proposal_sd = 0.8,
    jump_prob = 0.15,
    jump_sd = 1,
    seed = 100 + i
  )
})

jump_fraction_positive <- sapply(jump_chains, function(fit) {
  mean(fit$draws[(burn_in + 1):n_iter] > 0)
})

jump_results <- data.frame(
  start = starts,
  acceptance = round(sapply(jump_chains, `[`, "acceptance"), 3),
  fraction_positive = round(jump_fraction_positive, 3),
  mode_switches = sapply(jump_chains, `[`, "mode_switches")
)

jump_results
```

	start	acceptance	fraction_positive	mode_switches
1	-6	0.759	0.505	1577
2	-4	0.753	0.501	1516
3	4	0.757	0.509	1644

4 6 0.747 0.479 1558

The modified chains produced approximately (1516 – 1644) switches each and estimated fractions above zero close to 0.50, matching the symmetric target. This shows that the modified proposal moves efficiently between both modes rather than remaining trapped near one initial mode.

Exercise 5: MCMC as numerical integration

Consider the unnormalized density

$$\tilde{\pi}(x) = \exp\left(-\frac{x^4}{4} + \frac{x^2}{2}\right)$$

and

$$\mu = E_{\pi}\left\{\frac{1}{1+X^2}\right\}.$$

Task 1

Explain why Metropolis–Hastings does not require the normalizing constant.

Answer:

Suppose the normalized target is

$$\pi(x) = \frac{\tilde{\pi}}{Z}, \quad Z = \int \tilde{\pi} dx,$$

where Z is unknown.

The metropolis-Hastings acceptance probability is

$$\alpha(x, x^*) = \min\left\{1, \frac{\pi(x^*)q(x | x^*)}{\pi(x)q(x^* | x)}\right\}.$$

Substituting

$$\pi(x) = \tilde{\pi}(x)/Z$$

,

$$\frac{\pi(x^*)}{\pi(x)} = \frac{\tilde{\pi}(x^*)/Z}{\tilde{\pi}(x)/Z} = \frac{\tilde{\pi}(x^*)}{\tilde{\pi}(x)}.$$

The unknown normalizing constant Z appears in both numerator and denominator and cancels.

Task 2

Use `integrate()` to obtain a high-accuracy numerical benchmark for μ .

Answer:

Define the unnormalized target and the function whose expectation is required:

```
log_target <- function(x) {  
  -x^4 / 4 + x^2 / 2  
}  
  
unnormalized_target <- function(x) {  
  exp(log_target(x))  
}  
  
h_function <- function(x) {  
  1 / (1 + x^2)  
}
```

Compute the normalizing constant:

$$Z = \int_{-\infty}^{\infty} \tilde{\pi}(x) dx.$$

```
Z <- integrate(  
  unnormalized_target,  
  lower = -Inf,  
  upper = Inf,  
  subdivisions = 1000,  
  rel.tol = .Machine$double.eps^0.75  
)$value
```

Compute the numerator:

$$N = \int_{-\infty}^{\infty} \frac{1}{1+x^2} \tilde{\pi}(x) dx.$$

```

N <- integrate(
  function(x) h_function(x) * unnormalized_target(x),
  lower = -Inf,
  upper = Inf,
  subdivisions = 1000,
  rel.tol = .Machine$double.eps^0.75
)$value

```

Then the benchmark is

$$\mu = \frac{N}{Z}.$$

```

mu_benchmark <- N / Z

c(
  Z = Z,
  numerator = N,
  mu = mu_benchmark
)

```

```

      Z numerator      mu
3.9051372 2.3290282 0.5964011

```

The benchmark values are approximately

$$Z \approx 3.905, \quad N \approx 2.329 \quad \text{and} \quad \boxed{\mu \approx 0.596}$$

Task 3

Implement random-walk Metropolis. Choose a proposal scale using diagnostics rather than simply reporting the first value tried.

Answer:

```

rw_metropolis <- function(initial, n_iter, proposal_sd, seed = 1) {
  set.seed(seed)

  draws <- numeric(n_iter)
  draws[1] <- initial
  current <- initial
  n_accepted <- 0

  for (i in 2:n_iter) {
    proposal <- current + rnorm(1, mean = 0, sd = proposal_sd)

    log_ratio <- log_target(proposal) - log_target(current)

    if (log(runif(1)) < log_ratio) {
      current <- proposal
      n_accepted <- n_accepted + 1
    }

    draws[i] <- current
  }

  list(draws = draws, acceptance = n_accepted / (n_iter - 1))
}

# run several chains and return a matrix with one chain per row
run_chains <- function(starts, n_iter, proposal_sd, seed_base) {
  draws <- matrix(NA_real_, nrow = length(starts), ncol = n_iter)
  acceptance <- numeric(length(starts))

  for (k in seq_along(starts)) {
    fit <- rw_metropolis(starts[k], n_iter, proposal_sd, seed_base + k)
    draws[k, ] <- fit$draws
    acceptance[k] <- fit$acceptance
  }

  list(draws = draws, acceptance = acceptance)
}

```

```

# Gelman-Rubin R-hat
rhat <- function(chains) {
  chain_means <- rowMeans(chains)
  chain_vars <- apply(chains, 1, var)

  n_iter <- ncol(chains)
  within <- mean(chain_vars)
  between <- n_iter * var(chain_means)

  var_hat <- (1 - 1 / n_iter) * within + between / n_iter
  sqrt(var_hat / within)
}

# non-overlapping batch-means standard error
batch_se <- function(chains, batch_size) {
  batch_means <- apply(chains, 1, function(x) {
    h <- h_function(x)
    n_batches <- floor(length(h) / batch_size)
    h <- h[1:(n_batches * batch_size)]
    colMeans(matrix(h, nrow = batch_size))
  })

  sd(as.vector(batch_means)) / sqrt(length(batch_means))
}

# tune proposal scales
tune_n <- 20000
tune_burn <- 2000
proposal_grid <- c(0.5, 0.8, 1.0, 1.2, 1.5, 2.0, 2.5)

tuning_results <- do.call(rbind, lapply(proposal_grid, function(s) {
  run <- run_chains(
    starts = starts,
    n_iter = tune_n,
    proposal_sd = s,
    seed_base = 200
  )
})

```

```

retained <- run$draws[, (tune_burn + 1):tune_n]

data.frame(
  proposal_sd = s,
  acceptance = mean(run$acceptance),
  mu_estimate = mean(h_function(retained)),
  Rhat = rhat(retained),
  batch_se = batch_se(retained, batch_size = 500)
)
}))

print(tuning_results, digits = 4)

```

	proposal_sd	acceptance	mu_estimate	Rhat	batch_se
1	0.5	0.8509	0.5970	1.001	0.002161
2	0.8	0.7764	0.5954	1.001	0.001525
3	1.0	0.7316	0.5961	1.000	0.001379
4	1.2	0.6867	0.5968	1.000	0.001367
5	1.5	0.6247	0.5979	1.000	0.001532
6	2.0	0.5333	0.5998	1.000	0.001530
7	2.5	0.4570	0.5975	1.000	0.001760

```

# select proposal scale using the tuning diagnostics
selected_sd <- tuning_results$proposal_sd[which.min(tuning_results$batch_se)]
selected_sd

```

```
[1] 1.2
```

The proposal scale was selected by running four chains for several proposal standard deviations and comparing acceptance rate, $\hat{R}$, and batch-means standard errors. A proposal SD of 1.2 gave $\hat{R} \approx 1$, and acceptance rate around 0.69, and the smallest batch standard error.

```

# final chains for plotting and estimation
final_n <- 50000
final_burn <- 5000

final_run <- run_chains(

```

```

    starts = starts,
    n_iter = final_n,
    proposal_sd = selected_sd,
    seed_base = 300
  )

final_draws <- final_run$draws

# data frame for Task 3 plots
final_df <- data.frame(
  iteration = rep(seq_len(final_n), times = length(starts)),
  x = as.vector(t(final_draws)),
  start = factor(rep(starts, each = final_n))
)

palette <- c("springgreen4", "tan2", "steelblue", "tomato")

# trace plots
p_trace <- ggplot(final_df, aes(x = iteration, y = x, colour = start)) +
  geom_line(linewidth = 0.1) +
  facet_wrap(~ start, ncol = 2) +
  scale_colour_manual(name = "Initial value", values = palette) +
  labs(x = "Iteration", y = "X", title = "Random-walk Metropolis trace plots") +
  theme_pubr(base_size = 10, border = TRUE) +
  theme(
    panel.grid = element_blank(),
    strip.background = element_blank(),
    plot.title = element_text(size = 11, hjust = 0.5),
    legend.position = "top"
  )

# density plot
normalized_pdf <- function(x) {
  unnormalized_target(x) / Z
}

p_density <- ggplot(final_df, aes(x = x)) +
  geom_histogram(

```

```

aes(
  y = after_stat(density),
  fill = start,
  colour = start
),
bins = 100,
position = "identity",
alpha = 0.08,
linewidth = 0.1
) +
stat_function(
  fun = normalized_pdf,
  colour = "black",
  linetype = "dashed",
  linewidth = 0.6,
  inherit.aes = FALSE
) +
scale_fill_manual(name = "Initial value", values = palette) +
scale_colour_manual(name = "Initial value", values = palette) +
labs(
  x = "X",
  y = "Density",
  title = "Empirical densities and normalized target",
  subtitle = paste0("Selected proposal SD = ", selected_sd)
) +
theme_pubr(base_size = 10, border = TRUE) +
theme(
  panel.grid = element_blank(),
  strip.background = element_blank(),
  plot.title = element_text(size = 11, hjust = 0.5),
  legend.position = "top"
)

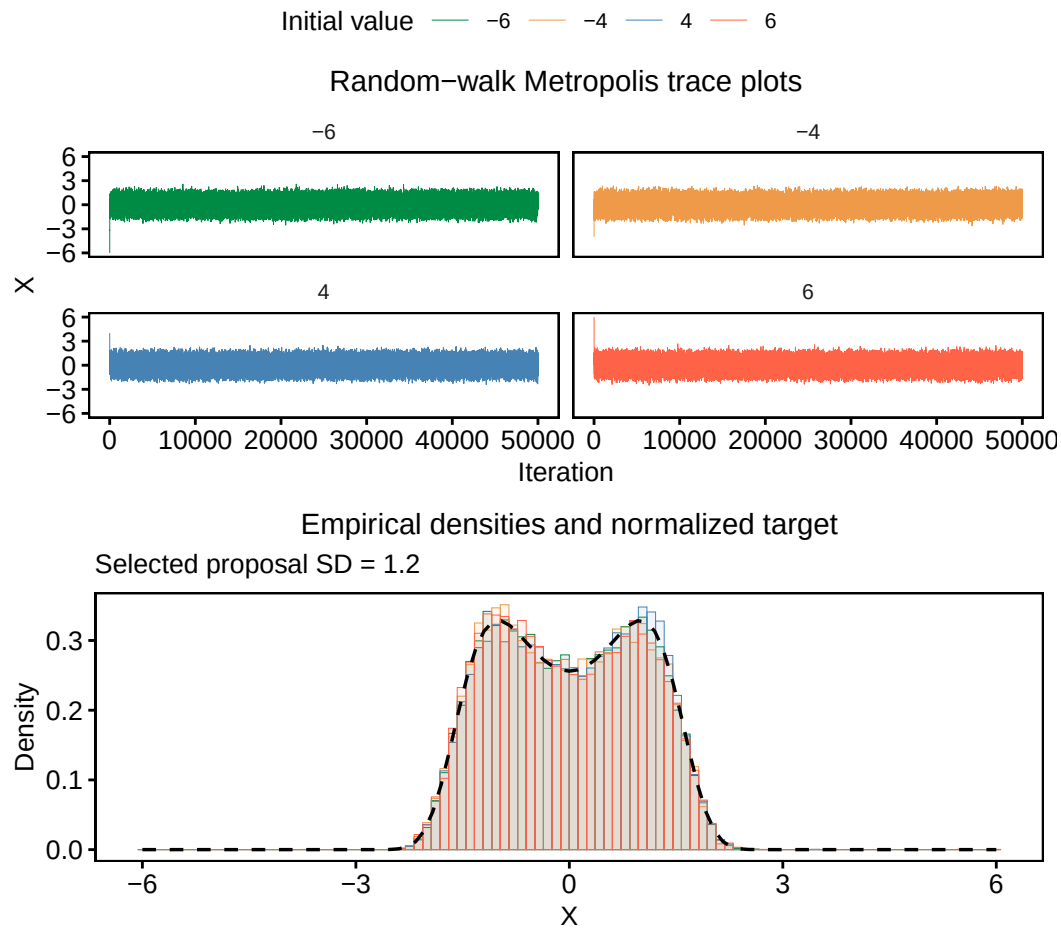
# combine
ggarrange(
  p_trace,
  p_density,
  nrow = 2,

```

```

common.legend = TRUE,
legend = "top"
)

```



Task 4

Estimate μ from retained MCMC draws.

Answer:

```

retained_draws <- final_draws[, (final_burn + 1):final_n]
h_mat <- h_function(retained_draws)

mu_mcmc <- mean(h_mat)

# per-chain estimates as a convergence check

```

```
chain_mu_table <- data.frame(
  start = starts,
  chain_mu = rowMeans(h_mat)
)
```

```
chain_mu_table
```

```
   start chain_mu
1     -6 0.5975490
2     -4 0.5962362
3      4 0.5950287
4      6 0.5972750
```

```
mu_mcmc
```

```
[1] 0.5965222
```

Per chain estimates are shown to verify that all chains sample the same distribution. The final value is

$$\hat{\mu}_{mcmc} \approx 0.59652,$$

and the benchmark value is

$$\hat{\mu}_{benchmark} \approx 0.5964.$$

They are close.

Task 5

Estimate the Monte Carlo standard error in a way that accounts for serial dependence. Compare it with the actual difference from the numerical benchmark. Explain why $\text{sd}(h(x))/\sqrt{M}$ is generally inappropriate for MCMC output.

Answer:

If the draws were independent, the standard error would be

```
iid_se <- sd(as.vector(h_mat)) / sqrt(length(h_mat))
```

For the representative run,

$$M = 180000, \quad SE_{iid} \approx 5.77 \times 10^{-4}.$$

Split each retained chain into batches of length 1,000 and calculate the mean of $h(X_t)$ within each batch:

```
mcmc_se <- batch_se(retained_draws, batch_size = 1000)
```

This gives

$$SE_{batch} \approx 9.52 \times 10^{-4}.$$

Compare with the benchmark

```
actual_difference <- abs(mu_mcmc - mu_benchmark)
z_score <- actual_difference / mcmc_se

comparison_table <- data.frame(
  quantity = c(
    "Numerical benchmark",
    "MCMC estimate",
    "Absolute difference",
    "IID-style SE",
    "Batch-means SE",
    "Difference / batch SE"
  ),
  value = c(
    mu_benchmark,
    mu_mcmc,
    actual_difference,
    iid_se,
    mcmc_se,
    z_score
  )
)
```

```
comparison_table
```

	quantity	value
1	Numerical benchmark	0.5964011245
2	MCMC estimate	0.5965222165
3	Absolute difference	0.0001210920
4	IID-style SE	0.0005774101
5	Batch-means SE	0.0009522262
6	Difference / batch SE	0.1271673044

The actual difference is about $\frac{\text{actual difference}}{\text{MCMC standard error}} \approx 0.1$ batch standard errors, which is plausible for a finite Monte Carlo run.

The naive formula $\frac{sd(h(X))}{\sqrt{M}}$ assumes that all M draws are independent and identically distributed. MCMC draws are usually serially correlated, and here $h(X_t)$ has positive lag-1 autocorrelation (see below).

```
lag1_autocorrelation <- apply(retained_draws, 1, function(x) {  
  h <- 1 / (1 + x^2)  
  cor(h[-length(h)], h[-1])  
})  
mean(lag1_autocorrelation)
```

```
[1] 0.4047772
```

Positive dependence means that each new draw adds less information than an independent draw, so the effective sample size is smaller than M .

Therefore, the naive formula generally underestimates Monte Carlo uncertainty. Batch means are more appropriate in this case, as they account for the serial dependence.

```
# running estimate plot  
running_df <- data.frame(  
  iteration = (final_burn + 1):final_n,  
  running_estimate = cumsum(colMeans(h_mat)) / seq_len(ncol(h_mat))  
)
```

```

p_running <- ggplot(running_df, aes(x = iteration, y = running_estimate)) +
  geom_line(colour = "springgreen4", linewidth = 0.5) +
  geom_hline(
    yintercept = mu_benchmark,
    colour = "tomato",
    linetype = "dashed",
    linewidth = 0.5
  ) +
  labs(x = "Iteration", y = "Running average of h(X)", title = "Convergence of the MCMC estimator") +
  theme_pubr(base_size = 10, border = TRUE) +
  theme(panel.grid = element_blank(),
        plot.title = element_text(size = 11, hjust = 0.5))

# Batch SE plot
batch_sizes <- c(250, 500, 1000, 2000)

batch_df <- data.frame(
  batch_size = factor(batch_sizes),
  batch_se = sapply(batch_sizes, function(b) {
    batch_se(retained_draws, b)
  })
)

p_batch <- ggplot(batch_df, aes(x = batch_size, y = batch_se, group = 1)) +
  geom_line(
    aes(colour = "Batch-means SE"),
    linewidth = 0.5
  ) +
  geom_point(
    aes(colour = "Batch-means SE"),
    size = 2
  ) +
  geom_hline(
    aes(yintercept = iid_se, colour = "IID-style SE"),
    linetype = "dashed",
    linewidth = 0.5
  ) +
  scale_colour_manual(

```

```

name = NULL,
values = c(
  "Batch-means SE" = "springgreen4",
  "IID-style SE" = "tomato"
)
) +
labs(
  x = "Batch size",
  y = "Monte Carlo standard error",
  title = "Batch SE diagnostics"
) +
theme_pubr(base_size = 10, border = TRUE) +
theme(
  panel.grid = element_blank(),
  plot.title = element_text(size = 11, hjust = 0.5),
  legend.position = c(0.78, 0.25),
  legend.background = element_blank()
)

ggarrange(p_running, p_batch, ncol = 2)

```

